

Lezione 2 02/10/2023 uml + class diagram

giovedì 5 ottobre 2023 10:55

Ricordiamo che anche java usa i puntatori per passaggio dei parametri : il passaggio dei parametri avviene per valore , ovvero la jvm alloca nuovo spazio in memoria (nello stack) ed il valore del parametro attuale viene copiato (o meglio il puntatore) nel parametro formale . **Da notare che in java non è possibile "giocare" con i puntatori** : non è possibile usare lo spiazamento dei puntatori esplicitamente, in quanto non vi è offset.

Cominciamo a vedere ora, cosa sono i linguaggi di modellazione: ovvero una rappresentazione di un qualunque oggetto ad un qualunque livello di astrazione, ognuno dei quali si sofferma su differenti scopi e dettagli. Ognuna di queste rappresentazioni , ha un **modello** che ne rappresenta le info necessarie , ma cosa è un modello e come viene usato? È una semplificazione di un sistema costruito con un determinato scopo . Si focalizza sugli aspetti per vedere se fa ciò per cui è stato costruito : quindi il modello viene creato per vedere se il sistema soddisfa gli obiettivi del sistema software. Per progettare un sistema software, in generale si usano 3 approcci : nessuno è migliore dell'altro : forma di artigianato nella quale tutta la conoscenza è basata su abilità personali e attitudini derivanti da studio ed esperienza, processo ingegnerizzato alla buona ovvero alcune delle fasi dello sviluppo in modo artigianale vengono svincolate da fattori umani mentre l'ultimo parte di un processo industriale , il quale si basa su interazione tra parti diverse ed è automatizzato, controllabile e ripetibile ovvero sfruttando meccanismi , riducendo così i costi di produzione, prevedendo prima le fasi ed uso delle metriche , sulle quali vengono basate le decisioni tale che il processo di sviluppo sw sia ripetibile . In generale i modelli sw sono utilizzati per sintetizzare ed analizzare i sistemi sw. In generale un sistema sw non è fatto solo di linee di codice ma anche di formati di dati (indipendente dalla scrittura del sw), regole di accesso ai dati , ruoli ed operazioni nel sistema , meccanismi/viste per rappresentare /raggiungere un obiettivo . Come già detto un sistema sw si basa su molteplici aspetti : funzionali (requisiti utente, requisiti implementativi norme o leggi), extra-funzionali (qualità servizio offerti e vincoli sulle risorse) ed organizzativi (aspetti tecnologici e/o non) ognuno dei quali deve essere trattato in modo opportuno. Andiamo a vedere un linguaggio : **UML(unified model language)** il quale fornisce supporto analisi , progettazione, programmazione e controllo evoluzione del sw . Ha una propria sintassi , strumenti per rappresentare cose, offre meccanismi di estensibilità ed astrae dal linguaggio di programmazione . Si basa sui concetti di Boock, Harel e Jacobson . Quindi in generale uml è un linguaggio di rappresentazione con standard industriali per visualizzare (sintassi visuale molto utile interazioni uomo-uomo) , specificare , costruire e documentare (documentazione sul sistema) i sistemi sw. Vediamo un pò di storia : inizialmente i modelli uml erano usati per forma di documentazione soprattutto nei casi d'uso / requisiti e su soluzioni progettuali , mentre per quello che riguarda i tool uml per pianificare attività, documentare fasi sviluppo e progettazione . Successivamente questi modelli sono stati usati per progettazione, sviluppo, testing (pianificazione e manutenzione) e documentazione : prodotti di ogni attività sono modelli che documentano e tracciano le soluzioni adottate. Quindi questi modelli uml servono anche per generare skeleton o pattern di esecuzione e permettono di risalire a modelli, partendo da pattern di codice : quindi il vero contributo di uml nella creazione /sviluppo processo sw è che lo permette attraverso fasi automatiche. I modelli uml si dividono in 3 categorie : **Structure** (pezzi del sistema e come sono collegati tra loro) : CLASS DIAGRAM , **behavioural** (comportamento del sistema) ed **interaction** (interazioni tra "pezzi" del sistema : STATE,ACTIVITY,USE CASE,SEQUENCE DIAGRAM. In generale si ha che il processo di sviluppo del sw non è sempre lineare , non ha un unico livello di astrazione , quindi non è possibile catturare e rappresentare tutti le informazioni con un singolo modello. Secondo uml il processo di sviluppo viene effettuato attraverso 5 fasi, ognuna delle quali cattura una serie di informazioni , ognuna delle quali enfatizza specifici aspetti dello sviluppo : **USE CASE , LOGICAL , IMPLEMENTATION, PROCESS , DEPLOYMENT** : in ognuna di queste fasi è possibile usare qualunque diagramma / modello .

1. Use case view : digramma che mostra le interazioni tra attore (utilizzatore del sistema) e sistema . Specifica cosa fa il sistema, analizzando i requisiti del sistema . Ha un livello di astrazione molto alto. Quindi gli utenti che sono impattati sono : clienti, progettisti, sviluppatori, tester (test di validazione) .
2. Logical view : enfatizza la struttura logica del sistema : si focalizza sul come . Data la conoscenza delle funzionalità (use case), si cerca di capire il come. Utenti impattati : progettisti e sviluppatori
3. Implementation view : modelli che enfatizzano come deve essere sviluppata la soluzione , organizzando il codice in moduli, eseguibilità e modelli specifici per ambiente di esecuzione. Utenti impattati sviluppatori, progettisti (attraverso feedback) e tester (basso livello).
4. Process view : modelli che descrivono gli "agganci" con le altre parti del sistema : processi da eseguire ed entità che eseguono i processi. Questo modello si usa per utilizzo efficace delle risorse, stabilire ordine esecuzione parallela degli oggetti e gestione eventi asincroni . Basato su modelli che descrivono la dinamica del sistema. Utenti impattati : sviluppatori ed integratori di sistema.
5. Deployment view : modelli che descrivono la topologia ed organizzazione delle macchine fisiche : quindi mappatura sistema sw su architettura fisica . Utenti impattati : sviluppatori, integratori e tester.

Sorge una domanda : quanto tempo usare una vista piuttosto che altra ? Come passo da una vista ad altra? La risposta la dà la "I" di uml : dato che è un linguaggio / modello non prescrive nessun processo di sviluppo per il sw , usando una qualunque metodologia di sviluppo : cascata, iterativo, spirale.

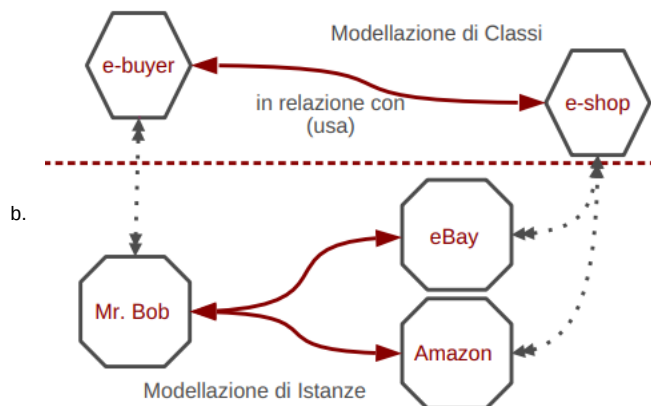
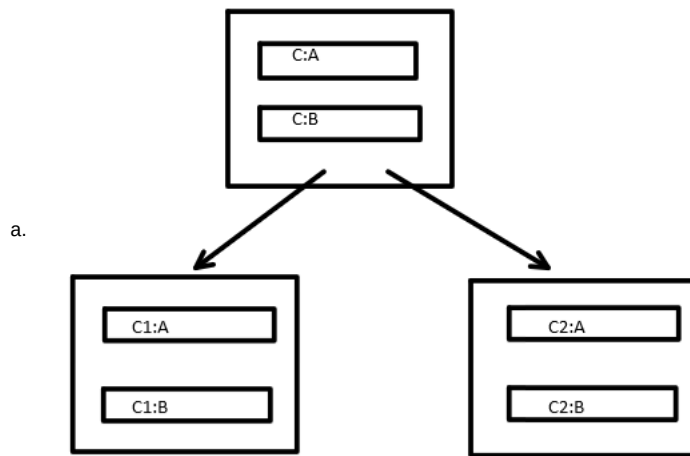
Addentriamoci nei concetti fondamentali della OO(Object orientation) :

classe,istanza,metodo,operazione,messaggio,incapsulamento,ereditarietà e polimorfismo. I primi 3 sono fondamentali, mentre gli ultimi 2 sono necessari per avere oo. Da notare che spesso il termine "object" viene usato sia per indicare una classe od una istanza (object è imbroglia, ma accettato).Andiamo ora vedere nel dettaglio :

1. Classe : rappresentazione/astrazione che rappresenta una famiglia di entità del dominio di applicazione (ruolo/attività). Rappresenta un elemento artificiale introdotto dal progettista. Caratterizzata da 2+1 caratteristiche fondamentali : proprietà (cosa posso fare : caratteristiche specifiche) ,comportamento (operazioni che la classe mette a disposizione: cosa posso fare/chiedere) e nome. Attenzione alla **coesione** : misura affinità tra elementi

utili per raggiungere uno stesso e determinato scopo. Si deve cercare di evitare di pensare alle classi come un modo procedurale : classi functoid .

2. Istanza : elemento del mondo reale . Rappresenta particolare valore che le variabili afferenti al tipo di dato della classe possono assumere in un determinato momento. Condividono comportamenti/proprietà e relazioni definite dalla classe. Definiscono quindi uno stato ed un comportamento.



3. Operazione : descrizione in modo astratto di una funzionalità offerta da una classe (anche parametrica) : funzionalità che può essere usata per leggere o scrivere le proprietà della classe. Quindi si ha accoppiamento tra intento e come viene realizzata. Specifica un prototipo/segnatura che un oggetto mette a disposizione di altri oggetti.
4. Metodo: porzione di codice che serve ad implementare un'operazione . Quindi esprime come viene effettivamente effettuata l'operazione , attraverso un'ottica procedurale.
5. Messaggio : richiesta di invocazione a run-time da parte di oggetto A su metodo da oggetto B.

Vediamo ora due tipi di approcci allo sviluppo del sw : **ottica procedurale ed oo**. Nel primo il comportamento del sistema si basa in funzione dello scheletro del diagramma di flusso : quindi partendo dal main si svolgono una serie di passi (anche ripetuti nel tempo) , modificando così una sequenza di variabili condivise globali. Nel secondo invece ogni oggetto/istanza ha i suoi dati , operazioni (utilizzate per modificare i dati). Gli attributi/proprietà allocate al momento della creazione e sono uniche! (allocazione di spazio in memoria) e rimangono fino alla de allocazione dell'oggetto (garbage collector). Due o più scatole (istanze) si possono scambiare messaggi. Quindi in quest'ottica si ragiona sugli elementi del dominio, proprietà e relazioni tra elementi.

Andiamo ora a vedere in dettaglio il class diagram: fa parte degli structural diagram e rappresenta le varie parti del sistema e le interazioni tra loro , quindi una sorta di organizzazione logica . Viene rappresentato attraverso un diagramma a grafo , quindi nodi (classi con nome, attributi e operazioni) + relazione (archi : possibili relazioni/interazioni tra le classi del sistema) . Se vi è presente un arco, la classe A può interagire con classe B o meglio una istanza di A può interagire con un a o più istanze di B e questa interazione dipende dal tipo di relazione.

